object_model/refinements_in_file.rb
```
module StringExtensions
  refine String do
    def to_alphanumeric
      gsub(/[^\w\s]/, '')
    end
  end
end
```

This code refines the String class with a new to_alphanumeric method. Differently from a regular Open Class, however, a Refinement is not active by default. If you try to call String#to_alphanumeric, you'll get an error:

```
"my *1st* refinement!".to_alphanumeric
```

```
NoMethodError: undefined method `to_alphanumeric' [...]
```

To activate the changes, you have to do so explicitly, with the using method:

```
using StringExtensions
```

From the moment you call using, all the code in that Ruby source file will see the changes:

```
"my *1st* refinement!".to_alphanumeric  # => "my 1st refinement"
```

Starting from Ruby 2.1, you can even call using inside a module definition. The Refinement will be active until the end of the module definition. The code below patches the String#reverse method—but only for the code inside the definition of StringStuff:

object_model/refinements_in_module.rb
```
module StringExtensions
  refine String do
    def reverse
      "esrever"
    end
  end
end

module StringStuff
  using StringExtensions
  "my_string".reverse    # => "esrever"
end

"my_string".reverse      # => "gnirts_ym"
```

Refinements are similar to Monkeypatches, but they're not global. A Refinement is active in only two places: the refine block itself and the code starting